

Research Notes on Recurrent neural network

Recurrent neural network

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

>> Section 1

The backward RNN processes in the opposite direction: $f_{\theta}''(x_N, h_N') = (y_{N-1}', h_{N-2}')$, $f_{\theta}''(x_{N-1}, h_{N-1}') = (y_{N-2}', h_{N-3}')$, ... $\{ \displaystyle f_{\theta}''(x_{\{N\}}, h_{\{N\}}') = (y_{\{N\}}', h_{\{N-1\}}'), f_{\theta}''(x_{\{N-1\}}, h_{\{N-1\}}') = (y_{\{N-1\}}', h_{\{N-2\}}'), \dots \}$

>> Section 2

When the neural network has learned a certain percentage of the training data. When the minimum value of the mean-squared-error is satisfied. When the maximum number of training generations has been reached. The fitness function evaluates the stopping criterion as it receives the mean-squared error reciprocal from each network during training. Therefore, the goal of the genetic algorithm is to maximize the fitness function, reducing the mean-squared error. Other global (and/or evolutionary) optimization techniques may be used to seek a good set of weights, such as simulated annealing or particle swarm optimization.

>> Section 3

A bidirectional RNN (biRNN) is composed of two RNNs, one processing the input sequence in one direction, and another in the opposite direction. Abstractly, it is structured as follows: The forward RNN processes in one direction: $f_{\theta}(x_0, h_0) = (y_0, h_1)$, $f_{\theta}(x_1, h_1) = (y_1, h_2)$, ... $\{ \displaystyle f_{\theta}(x_{\{0\}}, h_{\{0\}}) = (y_{\{0\}}, h_{\{1\}}), f_{\theta}(x_{\{1\}}, h_{\{1\}}) = (y_{\{1\}}, h_{\{2\}}), \dots \}$

>> Section 4

In artificial neural networks, recurrent neural networks (RNNs) are designed for processing sequential data, such as text, speech, and time series, where the order of elements is important. Unlike feedforward neural networks, which process inputs independently, RNNs utilize recurrent connections, where the output of a neuron at one time step is fed back as input to the network at the next time step. This enables RNNs to capture temporal dependencies and patterns within sequences. The fundamental building block of RNN is the recurrent unit, which maintains a hidden state—a form of memory that is updated at each time step based on the current input and the previous hidden state. This feedback mechanism allows the network to learn from past inputs and incorporate that knowledge into its current processing. RNNs have been successfully applied to tasks such as unsegmented, connected handwriting recognition,

speech recognition, natural language processing, and neural machine translation. However, traditional RNNs suffer from the vanishing gradient problem, which limits their ability to learn long-range dependencies. This issue was addressed by the development of the long short-term memory (LSTM) architecture in 1997, making it the standard RNN variant for handling long-term dependencies. Later, gated recurrent units (GRUs) were introduced as a more computationally efficient alternative. In recent years, transformers, which rely on self-attention mechanisms instead of recurrence, have become the dominant architecture for many sequence-processing tasks, particularly in natural language processing, due to their superior handling of long-range dependencies and greater parallelizability. Nevertheless, RNNs remain relevant for applications where computational efficiency, real-time processing, or the inherent sequential nature of data is crucial.

>> Section 5

Two RNNs can be run front-to-back in an encoder-decoder configuration. The encoder RNN processes an input sequence into a sequence of hidden vectors, and the decoder RNN processes the sequence of hidden vectors to an output sequence, with an optional attention mechanism. This was used to construct state of the art neural machine translators during the 2014–2017 period. This was an instrumental step towards the development of transformers.

>> Section 6

RNNs come in many variants. Abstractly speaking, an RNN is a function f_{θ} of type $(x_t, h_t) \mapsto (y_t, h_{t+1})$, where

>> Section 7

Each weight encoded in the chromosome is assigned to the respective weight link of the network. The training set is presented to the network which propagates the input signals forward. The mean-squared error is returned to the fitness function. This function drives the genetic selection process. Many chromosomes make up the population; therefore, many different neural networks are evolved until a stopping criterion is satisfied. A common stopping scheme can be:

>> Section 8

Machine translation Robot control Time series prediction Speech recognition Speech synthesis Brain–computer interfaces Time series anomaly detection Text-to-Video model Energy forecasting Rhythm learning Music composition Grammar learning Handwriting recognition

Research Notes on Recurrent neural network

Recurrent neural network

Human action recognition Protein homology detection
Predicting subcellular localization of proteins Several
prediction tasks in the area of business process
management Prediction in medical care pathways
Predictions of fusion plasma disruptions in reactors
(Fusion Recurrent Neural Network (FRNN) code)

it would see $(y_1, \dots, y_k)$ in order to generate $\hat{y}_{k+1}$.

>> Section 9

$\sigma(x)$: Sigmoid of x e.g. $\sigma(x) = 1 / (1 + e^{-x})$.

>> Section 10

Further reading Mandic, Danilo P.; Chambers, Jonathon A. (2001). Recurrent Neural Networks for Prediction: Learning Algorithms, Architectures and Stability. Wiley. ISBN 978-0-471-49517-8. Grossberg, Stephen (2013-02-22). "Recurrent Neural Networks". Scholarpedia. 8 (2): 1888. Bibcode:2013SchpJ...8.1888G. doi:10.4249/scholarpedia.1888. ISSN 1941-6016. Recurrent Neural Networks. List of RNN papers by Jürgen Schmidhuber's group at Dalle Molle Institute for Artificial Intelligence Research.

>> Section 11

Echo state networks (ESN) have a sparsely connected random hidden layer. The weights of output neurons are the only part of the network that can change (be trained). ESNs are good at reproducing certain time series. A variant for spiking neurons is known as a liquid state machine.

>> Section 12

Teacher forcing An RNN can be trained into a conditionally generative model of sequences, aka autoregression. Concretely, let us consider the problem of machine translation, that is, given a sequence $(x_1, x_2, \dots, x_n)$ of English words, the model is to produce a sequence $(y_1, \dots, y_m)$ of French words. It is to be solved by a seq2seq model. Now, during training, the encoder half of the model would first ingest $(x_1, x_2, \dots, x_n)$, then the decoder half would start generating a sequence $(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_l)$. The problem is that if the model makes a mistake early on, say at $\hat{y}_2$, then subsequent tokens are likely to also be mistakes. This makes it inefficient for the model to obtain a learning signal, since the model would mostly learn to shift $\hat{y}_2$ towards y_2 , but not the others. Teacher forcing makes it so that the decoder uses the correct output sequence for generating the next entry in the sequence. So for example,